



Instituto Tecnológico y de Estudios Superiores de Monterrey

Escuela de Ingeniería y Ciencias

Ingeniería en Ciencia de Datos y Matemáticas

Etapas 1 del reto: Gestor de Identidades

Equipo de Trabajo 2 - Grupo 603:

Valeria Arciga Valencia - A01737555

Henning Arvid Ladewig - A01831983

Alberto Boughton Reyes - A01178500

Ximena Montes Bautista - A01737949

Diego Octavio Arias Incháustegui - A00838285

Fernando Daniel Saucedo Hernández - A01385490

Profesores del bloque:

Raúl Gómez Muñoz, Daniel Otero Fadul,
Alberto Francisco Martínez Herrera, Pedro Leonel Olaya Trejos

Socio Formador:

Casa Monarca Ayuda Humanitaria al Migrante A.B.P.

Monterrey, Nuevo León

15 de mayo de 2026

Índice general

Abstract	4
1. Introducción	5
2. Marco de Referencia	5
2.1. Gestión de Identidades Digitales y Control de Acceso (IAM)	5
2.2. Criptografía y Confidencialidad de la Información	6
2.3. Metodología de Desarrollo Seguro (DevSecOps) y Control de Versiones	6
2.4. Arquitectura Tecnológica y Bases de Datos	6
2.5. Contexto Legal y Normativo	7
3. Desarrollo	7
3.1. Propuesta	7
3.1.1. Visión General	7
3.1.2. Justificación	8
3.1.3. Estructura de Archivos	10
3.2. Gestión de identidades y política de control de acceso	11
3.3. Desarrollo de componentes BackEnd	12
3.4. Desarrollo de FrontEnd	14
3.5. Conexión de los y flujos de operación	14
3.5.1. Flujos de Operación Principales	15
4. Resultados	17
5. Conclusiones y Trabajo Futuro	19
Referencias	20

Índice de cuadros

3.1. Matriz de compatibilidades e incompatibilidades tecnológicas del entorno de hosting. (elaboración propia).	9
3.2. Dependencias del sistema y descripción de paquetes de software (elaboración propia). . . .	10
3.3. Distribución de componentes del sistema y tecnologías (elaboración propia).	11
3.4. Matriz de permisos y niveles de acceso por perfil operativo (elaboración propia).	12
3.5. Estructura de Enums y valores permitidos para el control operativo (elaboración propia). . . .	13
4.1. Casos de prueba y enlaces de evidencia para los módulos de Usuarios y Certificados. (elaboración propia).	18

Índice de figuras

3.1. Arquitectura del sistema en capas con interconexión (elaboración propia).	8
3.2. Estructura de directorios y componentes del proyecto (elaboración propia).	11
3.3. Diagrama de flujo: Registro e inserción de nuevos usuarios en el sistema (elaboración propia).	16
3.4. Diagrama de flujo: Generación y almacenamiento de certificados criptográficos X.509 (elaboración propia).	16
3.5. Diagrama de flujo: Filtrado y consulta de certificados criptográficos mediante la API (elaboración propia).	17

Abstract

Este proyecto presenta el diseño e implementación de un Sistema de Gestión de Identidades Digitales para Casa Monarca Ayuda Humanitaria al Migrante A.B.P., organización de la sociedad civil que brinda apoyo legal, humanitario y educativo a personas migrantes en situación de vulnerabilidad en Nuevo León, México. Desarrollado bajo las restricciones de una infraestructura de alojamiento compartido, el sistema fue construido con Python (Flask), la biblioteca `cryptography` y TinyDB como capa de persistencia ligera.

La solución adopta una arquitectura cliente-servidor de tres capas e implementa criptografía de clave pública ($\text{RSA} \geq 2048$ bits), certificados digitales X.509 firmados con SHA-256 y cifrado simétrico AES para las claves privadas, garantizando los principios de Confidencialidad, Integridad y Disponibilidad (CID) de la información sensible de los beneficiarios.

Un modelo de Control de Acceso Basado en Roles (RBAC) con cuatro niveles jerárquicos rige todas las operaciones del sistema, mientras que una interfaz web administrativa permite la gestión del ciclo de vida de los certificados sin requerir conocimientos técnicos avanzados por parte del personal.

El sistema fue diseñado en cumplimiento de la Ley Federal de Protección de Datos Personales en Posesión de Particulares (LFPDPPP) y facilita el ejercicio de los Derechos ARCO. Las pruebas funcionales confirmaron la correcta operación de las funciones CRUD, los flujos criptográficos y los mecanismos de validación de entradas. La arquitectura modular permite una futura migración hacia un motor de base de datos relacional completo y una Infraestructura de Llave Pública (PKI) jerárquica.

1. Introducción

La migración forzada es un fenómeno a nivel global en el que millones de personas se ven obligadas a abandonar sus lugares de origen en búsqueda de seguridad, estabilidad y mejores oportunidades. Dentro de este contexto, Casa Monarca Ayuda Humanitaria al Migrante A.B.P., fungiendo como Organización Socio Formadora (OSF) dentro de la unidad de formación Uso de álgebras modernas para seguridad y criptografía, desempeña un papel esencial al brindar apoyo a migrantes en situación de vulnerabilidad mediante servicios como la asesoría legal, la ayuda humanitaria, la orientación laboral y el apoyo educativo.

Para llevar a cabo estas labores de asistencia, la organización requiere manejar información altamente sensible, incluyendo datos personales, documentos legales, constancias y registros médicos o jurídicos. Sin embargo, al igual que muchas organizaciones, Casa Monarca se enfrenta a limitaciones de recursos y barreras técnicas que dificultan la implementación de sistemas de seguridad informática avanzados, lo que expone esta información a un estado de vulnerabilidad. Al mismo tiempo que los retos técnicos que exige la situación problema, es importante que se cumpla con la legislación mexicana, la cual exige a toda organización garantizar el manejo y la protección adecuada de los datos personales.

En respuesta a esta problemática, el presente reto tiene como objetivo central desarrollar y proporcionar a Casa Monarca Ayuda Humanitaria al Migrante A.B.P. herramientas tecnológicas robustas que permitan garantizar que todos sus procesos digitales operen bajo los principios fundamentales de Confidencialidad, Integridad y Disponibilidad, logrando así beneficiar sus operaciones. Específicamente, la solución busca asegurar el estricto cumplimiento de la Ley Federal de Protección de Datos Personales en Posesión de Particulares (LFPDPPP) y facilitar el ejercicio de los Derechos ARCO (Acceso, Rectificación, Cancelación y Oposición) para los beneficiarios de la organización. Asimismo, el sistema está diseñado para dotar de veracidad, autenticidad y no repudio a los documentos que emita la OSF frente a terceros.

Para alcanzar estas metas, el proyecto propone la creación de un sistema integral de gestión de identidades basado en el uso de certificados digitales, el cual contará con las capacidades necesarias para dar de alta, revocar y rastrear de forma segura los accesos a la información.

Finalmente, para garantizar el éxito y la adopción operativa de la herramienta, la implementación se ha diseñado para ser totalmente transparente hacia los usuarios y colaboradores. El sistema busca no complicar la operación diaria ni exigir conocimientos técnicos avanzados al personal, enfocándose en mejorar los tiempos de atención y la productividad general. De esta forma, la solución entregada se proyecta como una plataforma fácil de usar, adaptada a los recursos actuales de la organización.

2. Marco de Referencia

Para el desarrollo del sistema de gestión de identidades y protección de datos para la organización Casa Monarca, fue fundamental analizar tanto el contexto legal mexicano como las tecnologías y metodologías de ciberseguridad actuales. Este capítulo detalla los conceptos, estándares y soluciones existentes en la literatura que sirvieron como base para el diseño y la toma de decisiones arquitectónicas del proyecto.

2.1. Gestión de Identidades Digitales y Control de Acceso (IAM)

El manejo de identidades digitales y el control de acceso (IAM) constituye un desafío tecnológico abordado a nivel global mediante diversas plataformas consolidadas. La identidad digital se define como "la representación única de un sujeto que participa en actividades en línea"[Grassi et al., 2017]. Su correcta gestión es un factor crítico, ya que, al tratarse de información altamente personal y sensible por razones socio políticas, una vulneración podría resultar en severos daños e impactos, tales como divulgación no autorizada de información confidencial, seguridad personal vulnerada o infracciones civiles o penales [Grassi et al., 2017].

En la industria, soluciones de código abierto como Keycloak ofrecen arquitecturas robustas para la gestión de usuarios, roles y autenticación multifactor en entornos empresariales. Esta plataforma funciona como un

intermediario utilizando protocolos estándar como OpenID Connect y SAML 2.0. Además, facilita el Single Sign-On (SSO) y gestiona la seguridad mediante el intercambio de JSON Web Tokens (JWT), garantizando la integridad en cada transacción. Sin embargo, la implementación de estos sistemas preconfigurados a menudo exige una infraestructura dedicada que excede las capacidades del alojamiento compartido que actualmente posee Casa Monarca. Por este motivo, la propuesta opta por un desarrollo adaptado a la medida de la Organización Socio Formadora. En este sentido, es imperativo establecer mecanismos que “limiten el acceso a los recursos de un sistema únicamente a los usuarios autorizados” [Stallings, 2017]. Adicionalmente, como señalan [Zúñiga and Hecht, 2023], los sistemas modernos requieren “mecanismos de autorización dinámicos y granulares que permitan asignar privilegios únicamente a los recursos necesarios y durante periodos de tiempo limitados”.

2.2. Criptografía y Confidencialidad de la Información

En el núcleo del sistema, el resguardo de la confidencialidad depende del uso de algoritmos de clave pública, tales como los esquemas de curvas elípticas, para proteger la generación y el intercambio de información. De manera complementaria, se integra el uso de funciones resumen (como el estándar SHA-256) para verificar la integridad de los datos almacenados y en tránsito [Zong, 2025]. Tras evaluar los requerimientos de la organización, se determinó que no se establecerán distinciones criptográficas ni esquemas de protección diferenciados por edad, garantizando que toda la información de la población atendida reciba el nivel máximo de protección posible sin sobrecargar el servidor. Para la implementación práctica, se emplean bibliotecas criptográficas establecidas como OpenSSL y frameworks en Python, los cuales son ampliamente usados en el desarrollo de algoritmos modernos. Estas herramientas permiten la implementación robusta de “generación de pares de claves, firmas digitales y validación de certificados en aplicaciones personalizadas” [Cryptography Project, 2026].

2.3. Metodología de Desarrollo Seguro (DevSecOps) y Control de Versiones

Para garantizar que el ciclo de vida del software sea seguro desde sus cimientos, el desarrollo se rige bajo la metodología DevSecOps. Esta práctica garantiza la integración continua de la seguridad desde las fases iniciales del diseño, permitiendo mitigar vulnerabilidades a través de herramientas de análisis preventivo [Mohammed et al., 2025, Nadipalli, 2023].

A nivel organizativo, la gestión de cambios se realiza mediante herramientas de control de versiones. Se implementa Git porque “es un sistema de control de versiones distribuido diseñado para gestionar todo tipo de proyectos, desde los más pequeños hasta los más grandes, con rapidez y eficiencia” [Chacon and Straub, 2014]. Esto se complementa con GitHub, plataforma que “con herramientas basadas en inteligencia artificial, pruebas de seguridad integradas y una integración perfecta, presta apoyo a los equipos desde el primer commit hasta el desarrollo empresarial” [GitHub, Inc., 2026], facilitando el trabajo colaborativo de manera clara y eficiente.

2.4. Arquitectura Tecnológica y Bases de Datos

La solución se ha modelado bajo una arquitectura cliente-servidor. El lenguaje de programación Python es la opción seleccionada para el backend debido a su alta compatibilidad con bibliotecas criptográficas y su amplio uso en el desarrollo de sistemas de seguridad [OpenSSL Project, 2023]. Finalmente, para la persistencia de datos, el sistema se apoya en una base de datos relacional para gestionar integralmente el registro de usuarios, las listas de revocación y la auditoría. MySQL destaca como una solución idónea gracias a su robusto control de concurrencia, mecanismos de seguridad integrada y compatibilidad nativa con los servidores del socio formador. Esta tecnología permite “administrar los privilegios de acceso mediante un sistema de cuentas de usuario y roles, poniendo a disposición del desarrollador un modelo de control de acceso flexible y seguro” [Oracle Corporation, 2024].

2.5. Contexto Legal y Normativo

El manejo de información perteneciente a poblaciones vulnerables, como las personas en situación de migración, trasciende los requerimientos puramente técnicos e impone obligaciones legales rigurosas. En el marco jurídico mexicano, la recolección, resguardo y procesamiento de esta información está regulada de manera estricta por la Ley Federal de Protección de Datos Personales en Posesión de Particulares (LFPDPPP). Esta legislación establece que toda organización (incluidas las asociaciones civiles y ONGs) debe garantizar un tratamiento legítimo, controlado e informado de los datos, a efecto de asegurar la privacidad y el derecho a la autodeterminación informativa de las personas [Cámara de Diputados del H. Congreso de la Unión, 20

Uno de los pilares de esta ley es la obligación de proporcionar mecanismos ágiles, gratuitos y transparentes para que los titulares de la información puedan ejercer sus Derechos ARCO (Acceso, Rectificación, Cancelación y Oposición). La arquitectura de gestión de identidades digitales propuesta para Casa Monarca responde directamente a esta exigencia legal. Al sentar las bases para la emisión y gestión del ciclo de vida de los certificados digitales, el sistema permite aislar compromisos de seguridad e inhabilitar accesos de manera trazable, garantizando el principio de responsabilidad y facilitando la Cancelación u Oposición.^{al} tratamiento de datos cuando el beneficiario o voluntario así lo solicite.

Adicionalmente, el proyecto aborda la necesidad de otorgar validez y no repudio a las constancias y documentos emitidos por la Organización Socio Formadora. La implementación de firmas y certificados basados en el estándar internacional X.509 dota de integridad y autenticidad a las operaciones electrónicas. Esto se alinea con los principios de fiabilidad y equivalencia funcional de los mensajes de datos establecidos en el Título Segundo del Código de Comercio y normas oficiales complementarias sobre conservación de mensajes de datos [Secretaría de Economía, 2017], asegurando que las herramientas tecnológicas entregadas a Casa Monarca cuenten con un respaldo probatorio ante terceros.

3. Desarrollo

En el presente capítulo se documentan a detalle las acciones realizadas para dar solución a la problemática planteada por la Organización Socio Formadora (OSF). A continuación, se expone la propuesta de solución, la arquitectura del sistema, la justificación de las decisiones tecnológicas adoptadas, así como el desglose de la implementación de los componentes del servidor (BackEnd), del cliente (FrontEnd) y la interconexión entre los mismos.

3.1. Propuesta

3.1.1. Visión General

Para resolver la problemática de Casa Monarca de administrar de forma centralizada y segura las identidades digitales de su personal abarcando desde profesionales y coordinadores hasta personal operativo y voluntarios, se propuso la construcción de un Sistema de Gestión de Identidades basado en la emisión y administración de certificados digitales X.509.

El sistema fue diseñado bajo un esquema de arquitectura cliente-servidor de tres capas (three-tier architecture), lo cual permite una separación lógica, modular y escalable de las responsabilidades operativas:

- Capa de Presentación (FrontEnd): Una interfaz web responsiva, orientada a ofrecer un panel administrativo intuitivo para la gestión y monitoreo del ciclo de vida de los certificados.
- Capa de Lógica (BackEnd / API): Una interfaz de programación de aplicaciones (API REST) responsable de procesar las reglas de negocio, validar los datos de entrada, garantizar la autenticación y ejecutar las operaciones criptográficas.
- Capa de Persistencia de Datos: Almacenamiento diseñado de manera desacoplada para facilitar el despliegue inicial del Producto Mínimo Viable (MVP) y permitir una migración transparente hacia un

motor relacional robusto en etapas posteriores.

En la figura 3.1 se detalla un diagrama de la arquitectura de 3 capas, en donde se observa la conexión entre el FrontEnd, BackEnd y el Almacenamiento de la información correspondiente.

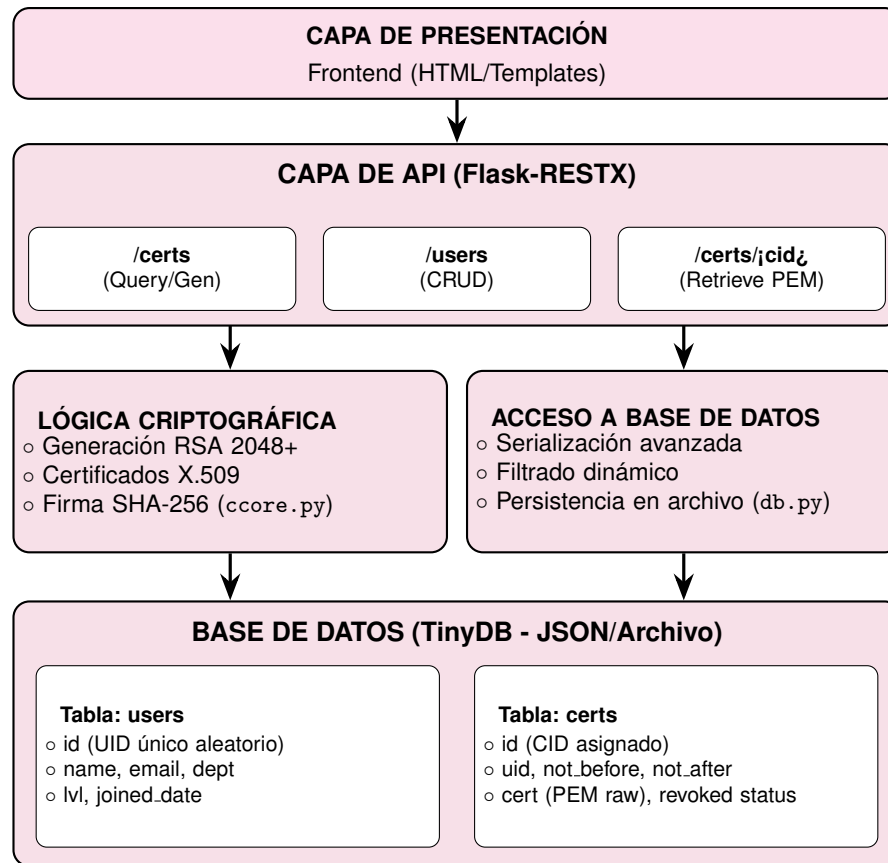


Figura 3.1: Arquitectura del sistema en capas con interconexión (elaboración propia).

3.1.2. Justificación

El diseño de la propuesta se fundamentó en un análisis exhaustivo de los requerimientos operativos de la organización y sus restricciones de infraestructura física. En particular, se evaluaron las capacidades del servicio de alojamiento compartido actual (*HostGator*) con el que cuenta Casa Monarca.

Para determinar la viabilidad del proyecto, se elaboró una matriz de compatibilidad (Cuadro 3.1), la cual expone las herramientas soportadas por el servidor y las incompatibilidades notables que restringen el uso de tecnologías empresariales preconfiguradas.

Categoría	Herramientas y Tecnologías Compatibles	Incompatibilidades Notables
Lenguajes y Frameworks	PHP: 8.0, 8.1, 8.2. Python: 3.6. Frameworks: Laravel (10 y 11), Django (solo hasta 3.2), Ruby on Rails, CakePHP. Otros: JavaScript, JSON, FastCGI, Perl 5, cURL, módulos PECL.	Node.js, Java (JSP/Tomcat), ASP.NET (todas las versiones), Zend Framework, PHP 5 y 7.
Bases de Datos	Soportadas: MySQL 5.7 (CentOS) o MySQL 8.0 (AlmaLinux), MySQLi, PDO (pdo_mysql).	PostgreSQL, MariaDB, Microsoft SQL Server, Oracle, MongoDB.
CMS y E-commerce	Gestores: WordPress (y WP MU), Drupal, Joomla (hasta 5.5.2). E-commerce: Magento (2.3), PrestaShop (1.7 y 8), OpenCart. E-learning: Moodle (hasta 4.1).	Versiones más recientes o no especificadas de Joomla y Moodle requerirán VPS.
Paneles y Servicios	Paneles: cPanel, WHM, Softaculous (instalador de apps). Accesos: FTP (Cyberduck), SSH2, WebDisk, Tareas Cron. Correo: Webmail (RoundCube), IMAP, POP3, SMTP, Exim.	Plesk, Docker, VPN, Servidores de juegos, Terminales como KVM o Remote Desktop.
Seguridad y SSL	Certificados: SSL Gratis (Let's Encrypt) para todos los dominios, Wildcard SSL, OpenSSL. Antispam: SpamAssassin.	SSL compartido, Certificación HIPAA compliance, PCI Compliance.
Otras Herramientas	Git (Cliente), SVN (Cliente), AWstats, ImageMagick, Archivos comprimidos (.zip, .tar.gz), Ajax, HTML.	FFmpeg, Live streaming, Load balancing, Redis/Memcached, XAMPP.

Cuadro 3.1: Matriz de compatibilidades e incompatibilidades tecnológicas del entorno de hosting. (elaboración propia).

El análisis de esta matriz determinó directamente las decisiones de ingeniería para el Producto Mínimo Viable (MVP). Al presentarse incompatibilidades con motores relacionales como PostgreSQL y entornos de ejecución como Node.js, el equipo optó por un desarrollo a la medida bajo el siguiente stack tecnológico:

- **Lenguaje y Framework Base:** Se seleccionó Python 3.6 (versión máxima soportada nativamente) utilizando el micro-framework Flask (v. 1.1.2) por su bajo costo computacional e ideal para el despliegue de prototipos funcionales y APIs con alta velocidad. A diferencia de arquitecturas más pesadas, Flask proporcionó la flexibilidad necesaria sin imponer consumo excesivo de memoria. Adicionalmente, la extensión Flask-RESTX facilitó la estandarización de endpoints, la validación automática de esquemas de datos y la autogeneración de documentación interactiva a través de Swagger UI.
- **Motor Criptográfico Estandarizado (cryptography):** Para el núcleo de seguridad, se descartaron alternativas de bajo nivel en favor de la biblioteca cryptography (v. 3.4.7), consolidada como el estándar en Python. Esta selección permitió la generación eficiente de llaves asimétricas RSA (con tamaños superiores a los 2048 bits recomendados por el NIST), la estructuración de los certificados X.509 y el cifrado de las claves privadas en formato PEM mediante el estándar avanzado AES, permitiendo operar independientemente del servidor web.
- **Base de Datos (TinyDB):** Para la fase de validación técnica (MVP), se implementó TinyDB (v. 4.4.0), una base de datos ligera basada en archivos JSON que no requiere dependencias externas. Se puede destacar que el diseño de la capa de datos (db.py) se programó con un grado máximo de desacoplamiento, lo que garantiza que, cuando el sistema escale hacia un entorno de producción con bases relacionales completas, la API y el motor criptográfico no requerirán reestructuración.
- **Infraestructura de Llaves Públicas:** Considerando el alcance de la etapa inicial y el contexto corporativo interno de la OSF, se implementó una cadena de confianza fundamentada en certificados

autofirmados.

- **Desarrollo de FrontEnd:** Se determinó no integrar bibliotecas modernas de renderizado como React o Angular. Estos marcos de trabajo exigen pasos de compilación complejos, lo que incrementaría drásticamente la dificultad de mantenimiento para el personal de Casa Monarca. La interfaz se construyó empleando lenguajes nativos de la web y la API estándar fetch(), garantizando compatibilidad universal y estabilidad a largo plazo.

En el Cuadro 3.2 se detalla el conjunto final de dependencias de software declaradas en el archivo de configuración requirements.txt que conforman el núcleo tecnológico del sistema.

Paquete	Versión	Descripción Operativa
flask	1.1.2	Framework web ligero para la gestión del servidor HTTP.
flask-restx	0.3.0	Extensión Flask para la especificación y validación de endpoints REST.
flask-swagger-ui	3.36.0	Interfaz interactiva embebida para la documentación de la API.
flask-cors	3.0.10	Mecanismo para el control de acceso entre orígenes cruzados.
tinydb	4.4.0	Base de datos embebida basada en archivos JSON para persistencia local.
cryptography	3.4.7	Operaciones criptográficas avanzadas (X.509 y llaves RSA).
bcrypt	3.2.0	Biblioteca para funciones de hash seguro y manejo de contraseñas.
validators	0.18.2	Componente para la validación estricta de sintaxis de correos electrónicos y datos.

Cuadro 3.2: Dependencias del sistema y descripción de paquetes de software (elaboración propia).

3.1.3. Estructura de Archivos

Para garantizar el mantenimiento preventivo y la escalabilidad del sistema por parte del equipo de TI de la organización, el proyecto se organizó bajo una estructura de directorios modular. En la Figura 3.2 se detalla el árbol de archivos que componen la solución técnica.

Con el objetivo de auditar la solución, se ha realizado una recopilación de los componentes del sistema, los cuales se detallan en el Cuadro 3.3.

```

MA2006B.Reto/
├── src/
│   ├── backend/
│   │   ├── main.py # Punto de entrada
│   │   ├── api_intf.py # Definición de endpoints
│   │   ├── ccore.py # Funciones criptográficas
│   │   ├── db.py # Acceso a datos
│   │   └── templates/
│   │       └── index.html # Frontend HTML
│   └── examples/
│       ├── create_dummy_db.py # Script de inicialización DB
│       ├── generate_server_cert.py # Genera clave admin
│       ├── generate_user_cert.py # Genera certificados usuario
│       ├── backend/ # Ejemplos adicionales
│       └── tls/ # Ejemplos TLS
├── requirements.txt # Dependencias
├── setup.py # Configuración del paquete
├── README.md # Documentación
└── .gitignore # Exclusiones git

```

Figura 3.2: Estructura de directorios y componentes del proyecto (elaboración propia).

Componente	Tecnología	Responsabilidad de Software
main.py	Flask 1.1.2	Inicialización del servidor, configuración CORS y carga de claves maestras.
api_intf.py	flask-restx	Definición de puntos de enlace (endpoints), esquemas y validación REST.
ccore.py	cryptography	Generación de llaves RSA, estructura de certificados X.509 y firma SHA-256.
db.py	TinyDB 4.4.0	Capa de persistencia de usuarios/certificados y rutinas de serialización.
index.html	HTML / CSS / JS	Interfaz de usuario, dashboard de métricas, control de modales y validación local.

Cuadro 3.3: Distribución de componentes del sistema y tecnologías (elaboración propia).

3.2. Gestión de identidades y política de control de acceso

El diseño de la solución contempla la dinámica de rotación de colaboradores en Casa Monarca, estructurando un modelo jerárquico para categorizar a los usuarios. Este modelo se implementó en el código a través de la enumeración `LevelEnum`, en donde se clasifica las identidades determinando qué perfiles requieren credenciales criptográficas y estableciendo sus límites de operación.

De acuerdo con las reglas de negocio establecidas para la Organización Socio Formadora, el flujo de la información exige un control estricto sobre quién registra, revisa y modifica los datos de los beneficiarios. Para ello, el sistema modela un Control de Acceso Basado en Roles (RBAC) fundamentado en las operaciones de bases de datos CRUD (Crear, Leer, Actualizar, Eliminar).

A nivel de código, esta jerarquía se encuentra parametrizada a través del enumerador `LevelEnum`, estableciendo cuatro niveles con capacidades escalonadas:

1. Nivel 1 - Administrador: Posee control absoluto sobre el sistema.
2. Nivel 2 - Coordinador: Personal con autoridad para la gestión de su área. El código refleja la segmen-

tación de este nivel mediante el DeptEnum, mapeando las áreas operativas.

3. Nivel 3 - Operativo: Personal enfocado en la revisión y canalización de datos.

4. Nivel 4 - Usuario / Captura: Registro inicial de beneficiarios (becarios, voluntarios).

A continuación, en el Cuadro 3.4, se sintetiza la matriz de acceso y la delegación de responsabilidades para cada perfil del sistema.

Nivel de Acceso	Perfil	Crear (C)	Leer (R)	Actualizar (U)	Eliminar (D)	Rol Operativo
Nivel 1	Administrador	✓	✓	✓	✓	Gestión total
Nivel 2	Coordinador	✓	✓	✓	×	Gestión de área funcional.
Nivel 3	Operativo	✓	✓	×	×	Revisión y canalización.
Nivel 4	Usuario / Captura	✓	×	×	×	Registro inicial de beneficiarios.

Cuadro 3.4: Matriz de permisos y niveles de acceso por perfil operativo (elaboración propia).

3.3. Desarrollo de componentes BackEnd

El servidor central (BackEnd) es el núcleo lógico y de seguridad del Sistema de Gestión de Identidades. Su desarrollo se estructuró en cuatro módulos fundamentales para garantizar una separación adecuada de responsabilidades: orquestación, interfaz de comunicación, operaciones criptográficas y persistencia de datos.

A continuación, se detalla la implementación técnica de cada componente:

Módulo de inicialización (`main.py`)

Este componente es el punto de entrada de la aplicación. Su responsabilidad principal es la de configurar e inicializar el servidor web utilizando el entorno de Flask.

Para la gestión de claves maestras, durante la fase de inicio, el script llama a una función de inicialización criptográfica para cargar de manera segura en memoria la clave privada del administrador, almacenada en un archivo formato `.pem`. Esta llave resulta crítica, pues actúa como la Autoridad Certificadora (CA) interna que firmará todos los certificados subsecuentes.

En cuanto a la configuración de red y documentación, se habilitó el soporte para el intercambio de Recursos de Origen Cruzado (CORS), permitiendo la comunicación fluida con la capa de presentación. Adicionalmente, se configuró la ruta base bajo el prefijo `/api` y se registró el plano para la interfaz gráfica en `/api/docs`, lo que expone la documentación interactiva de la API para desarrolladores.

Interfaz de programación de aplicaciones (`api_intf.py`)

Este módulo define la comunicación del sistema mediante una arquitectura RESTful, utilizando la extensión flask-restx. Se encarga de procesar las solicitudes entrantes, validar la integridad de las cargas útiles (payloads) y dar las respuestas HTTP correspondientes. Para este módulo se programaron enumeraciones estrictas para modelar la estructura organizacional de Casa Monarca. Se definieron los departamentos correspondientes a los existentes en Casa Monarca y los niveles de jerarquía dentro de la organización como se detalla en el Cuadro 3.5. Además, como Endpoint de identidades, se implementaron las operaciones CRUD (Crear, Leer, Actualizar, Eliminar). Las peticiones tipo GET soportan un motor de búsqueda por expresiones regulares (regex) y rangos de fechas. Para el registro (POST), se integraron validaciones

estrictas, asegurando, por ejemplo, que el formato del correo electrónico sea válido mediante la librería `validators` y que este sea único en todo el padrón de usuarios.

Estructura (Enum)	Valores Permitidos (Colección)	Propósito Operativo
DeptEnum	Humanitaria, PsicoSocial, Legal, Comunicación, Almacén, TI	Delimitación estricta de las áreas y departamentos funcionales de Casa Monarca.
LevelEnum	admin, coordinador, operativo, captura, voluntario	Asignación del nivel de acceso jerárquico asociado al ciclo de vida del usuario.

Cuadro 3.5: Estructura de Enums y valores permitidos para el control operativo (elaboración propia).

Por otro lado, como Endpoint de certificados (`/certs`), el sistema maneja el ciclo de vida de los identificadores X.509. Su método POST verifica que las fechas de vigencia solicitadas sean lógicas, en donde la fecha final sea posterior a la inicial y no en el pasado y que la longitud de la clave solicitada no sea inferior a 2048 bits por normativa de seguridad.

Núcleo criptográfico (`ccore.py`)

Es el componente más robusto en términos computacionales, encargado de ejecutar las primitivas criptográficas mediante la biblioteca `cryptography`. La función principal, `create_cert()`, centraliza la generación de las identidades digitales operando en tres fases secuenciales. En primer lugar, se hace la generación de claves asimétrica RSA. Se estableció por defecto un tamaño de clave de 2048 bits (configurable hasta 4096 bits) y un exponente público de Fermat (65537) para optimizar la eficiencia de las operaciones de cifrado.

Como segunda fase, la construcción del estándar X.509, donde se instancia un objeto de certificado digital configurando el Sujeto (Subject) con los metadatos organizacionales fijos de Casa Monarca (País: MX, Estado: Nuevo León) y los datos dinámicos del usuario como son Nombre, Email, Departamento y Nivel inyectados en la Unidad Organizacional. Posteriormente, se asigna un número de serie criptográficamente aleatorio y se firman estos datos con la clave privada del administrador, utilizando la función de resumen seguro SHA-256.

Finalmente, como protección de clave privada, la clave generada para el usuario nunca se almacena en texto plano. Se serializa a formato PEM y se somete a un proceso de cifrado simétrico utilizando el algoritmo avanzado AES (Best Available Encryption) junto con una contraseña única temporal, garantizando que solo el propietario final pueda instalar la credencial, esto, con la intención de que el usuario tenga el archivo PEM dentro de un Universal Serial Bus (USB).

Capa de Datos (`db.py`)

Este módulo administra la persistencia de la información en memoria secundaria, interactuando de manera exclusiva con el motor TinyDB, esto al realizar modelado y serialización, dado que los objetos como fechas nativas de Python (`date`) o los certificados `x509.Certificate` no son compatibles directamente con estructuras JSON, se desarrollaron funciones bidireccionales de serialización y deserialización. Las fechas se transforman en tuplas de enteros (año, mes, día) y los certificados se decodifican a cadenas de texto PEM para su almacenamiento seguro.

En términos de integridad referencial y unicidad, las funciones de escritura (`add_user` y `add_cert`) están programadas con rutinas de validación interna. Al registrar un nuevo usuario o emitir un certificado, el sistema iterará utilizando generadores de números aleatorios hasta encontrar e indexar un Identificador Único de Usuario (UID) y un Identificador de Certificado (CID) que no existan previamente en las tablas, previniendo colisiones de datos y generando mensajes de error.

3.4. Desarrollo de FrontEnd

La capa de presentación (FrontEnd) fue diseñada con un enfoque centrado en la usabilidad y la eficiencia, garantizando que los administradores de Casa Monarca puedan gestionar el ciclo de vida de las identidades sin requerir conocimientos técnicos avanzados. Su construcción se basó enteramente en tecnologías web nativas (HTML5, CSS3 y JavaScript), conformando un único archivo principal (index.html) que centraliza la estructura, el diseño y la lógica de interacción. La arquitectura de la interfaz de usuario se divide en los siguientes componentes modulares.

En un primer lugar, el panel de control y métricas (Dashboard), compuesto por una cuadrícula superior compuesta por cuatro tarjetas estadísticas que proporcionan información sobre el estado del sistema. Este panel extrae la información del estado global y calcula el total de usuarios registrados, el volumen de certificados vigentes (cruzando los datos de validez contra la fecha actual del sistema), la cantidad de certificados revocados y una alerta temprana para aquellos próximos a expirar en un margen de 30 días.

Posteriormente se incluye una tabla dinámica de identidades, que es el punto central de la visualización de datos. Este componente hace posible la visualización del padrón cruzado de usuarios y certificados. Se programó con un motor de filtrado que permite realizar búsquedas por coincidencia de texto (nombre o correo), así como aplicar filtros categóricos por nivel de acceso o estado del certificado, actualizando el Modelo de Objetos del Documento (DOM) dinámicamente sin necesidad de recargar la página.

En cuanto al sistema de flujos emergentes, se ha desarrollado para aislar las operaciones críticas y evitar la pérdida de contexto, se implementaron interfaces superpuestas (overlays). Destacan el modal de creación (overlay-new), que unifica el formulario de registro y la configuración de vigencia criptográfica; el modal de edición (overlay-edit); y el visor de certificados (overlay-cert), el cual decodifica visualmente los atributos X.509, muestra una barra de progreso de vida útil y expone el bloque en formato PEM.

Finalmente, para la gestión de estado y peticiones asíncronas, toda la comunicación con el servidor se maneja a través de la *API fetch()*. Se implementó un estado global en el navegador (variables USERS y CERTS) que se sincroniza con la base de datos mediante la función `loadAll()` tras cada operación de escritura, garantizando que la tabla dinámica y los indicadores visuales siempre reflejen la información actual operativa del servidor.

3.5. Conexión de los y flujos de operación

La interconexión y la orquestación del Sistema de Gestión de Identidades operan mediante un modelo desacoplado de transferencia de estado representacional (REST). La comunicación externa entre la capa de presentación (*FrontEnd*) y la lógica de negocio (*BackEnd*) se realiza de forma asíncrona mediante peticiones HTTP estructuradas en formato JSON, garantizando un acoplamiento débil entre componentes. Internamente en el servidor, los módulos de la API, el motor criptográfico y la persistencia se integran de manera síncrona mediante llamadas nativas de funciones.

Con el objetivo de mitigar el ingreso de datos corruptos a la base de datos, el diseño implementa una estrategia de validación ejecutada tanto en la frontera del cliente como en la del servidor. A continuación, se detalla mecánicamente el ciclo de vida completo de los datos a través del flujo de extremo a extremo (*End-to-End*) para el registro de una identidad y la emisión de su credencial criptográfica X.509.

El proceso inicia en la capa de presentación cuando el administrador introduce los datos del colaborador en los formularios de interfaz gráfica. Antes de iniciar la transmisión, el motor ejecuta rutinas de validación de primera línea, en donde se bloquean campos obligatorios vacíos y la verificación de correos electrónicos. Si la entrada es válida y el nivel de acceso requiere credenciales, como en el caso de coordinadores, el sistema invoca a la API criptográfica (`crypto.getRandomValues`) para la generación de una contraseña pseudoaleatoria con una longitud fija de 20 caracteres en texto plano.

En cuanto a la persistencia de la identidad base, el motor serializa los metadatos e inicia una petición síncrona mediante el método `POST` hacia el endpoint `/api/users`. El módulo `api_intf.py` intercepta la carga útil, valida el esquema JSON y delega el almacenamiento al componente de persistencia invocando a `db.add_user()`.

Esta función valida que la dirección de correo electrónico sea única en el sistema. En caso de que no existan colisiones, se calcula mediante un ciclo un Identificador Único de Usuario (UID) aleatorio de tipo entero, transforma la fecha nativa a una tupla almacenable, escribe el registro en el archivo JSON de TinyDB y retorna un código de estado HTTP 201 junto con el UID generado.

Para la solicitud de emisión criptográfica, al recibir la confirmación del registro base, la lógica del cliente encadena de forma programática y transparente para el operador una segunda petición mediante el método POST hacia el endpoint `/api/certs`. En esta solicitud, el cliente inyecta el UID recién obtenido como un parámetro de consulta (*Query Parameter*), mientras que el tamaño de la llave y la contraseña simétrica local se transmiten de manera segura en el cuerpo del mensaje (*Request Body*).

Hacia el procesamiento de claves y firma digital, el método `api_intf.Certs.post()` recibe el paquete, valida que los rangos de fechas no pertenezcan al pasado y que la fecha de expiración sea estrictamente posterior a la de inicio, y que la longitud de la clave RSA sea igual o superior a 2048 bits.

Posteriormente, invoca al núcleo criptográfico en `ccore.create_cert()`. Este componente computa algebraicamente el par de llaves asimétricas utilizando un exponente público estándar de Fermat (65537), cifra la clave privada del usuario en formato PEM aplicando el algoritmo simétrico AES bajo la contraseña generada en el paso 1, y estructura el certificado digital X.509 v3 inyectando los atributos jerárquicos del colaborador firmados mediante el algoritmo de resumen seguro SHA-256 con la clave privada de la Autoridad Certificadora (CA) maestra cargada en `main.py`.

Acerca del almacenamiento, el certificado público generado se transfiere a la función `db.add_cert()`, la cual genera un Identificador de Certificado (CID) único, convierte el objeto criptográfico a una representación de texto plano codificada en PEM y añade el registro vinculándolo al UID del propietario con una bandera de revocación inicializada en falso (`revoked: false`). El servidor concluye la operación respondiendo al cliente con un estado HTTP 201 y una carga útil que contiene el CID y la clave privada cifrada.

Una vez que el navegador captura la respuesta exitosa del servidor, el *FrontEnd* desencadena de forma automatizada la descarga local de la clave privada cifrada en un archivo con extensión `.pem`. Simultáneamente, despliega un cuadro de diálogo emergente de un solo uso que expone la contraseña de descifrado en texto plano para que sea entregada al usuario.

Tras el cierre del modal, el cliente ejecuta la función global `loadAll()` enviando peticiones GET para recuperar el padrón actualizado desde la API, actualizando el Modelo de Objetos del Documento (DOM) de la tabla administrativa y actualizando las métricas del tablero principal.

3.5.1. Flujos de Operación Principales

Para complementar la descripción del comportamiento dinámico del sistema y detallar detalladamente la secuencia lógica programada en las capas del servidor para cada uno de los endpoints de la API REST, se presentan a continuación los diagramas de flujo técnicos que gobiernan el BackEnd.

En primer lugar, la Figura 3.3 ilustra de manera lineal el proceso síncrono que se desencadena al dar de alta una nueva entidad dentro del endpoint de usuarios del sistema.

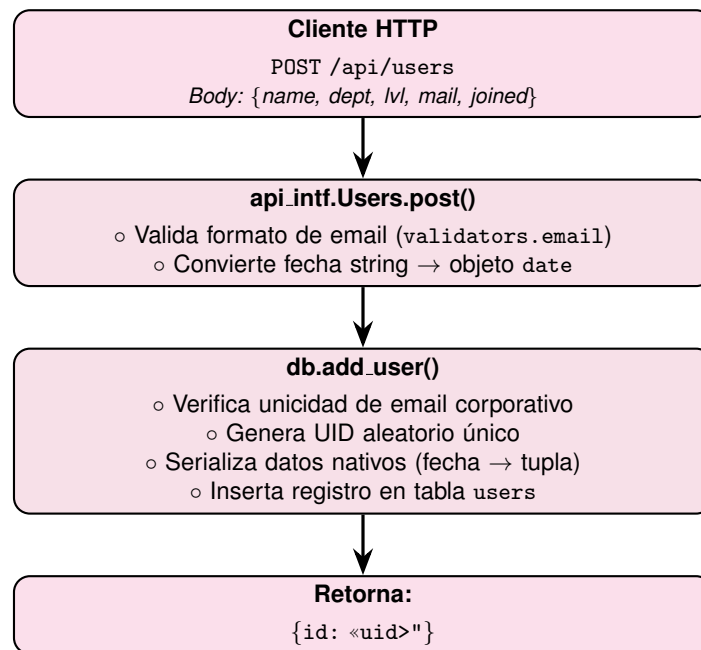


Figura 3.3: Diagrama de flujo: Registro e inserción de nuevos usuarios en el sistema (elaboración propia).

Posteriormente, en la Figura 3.4 se describe la lógica computacional del endpoint de certificados encargado de la derivación asimétrica de claves RSA y la estructuración del estándar de identidad firmado.

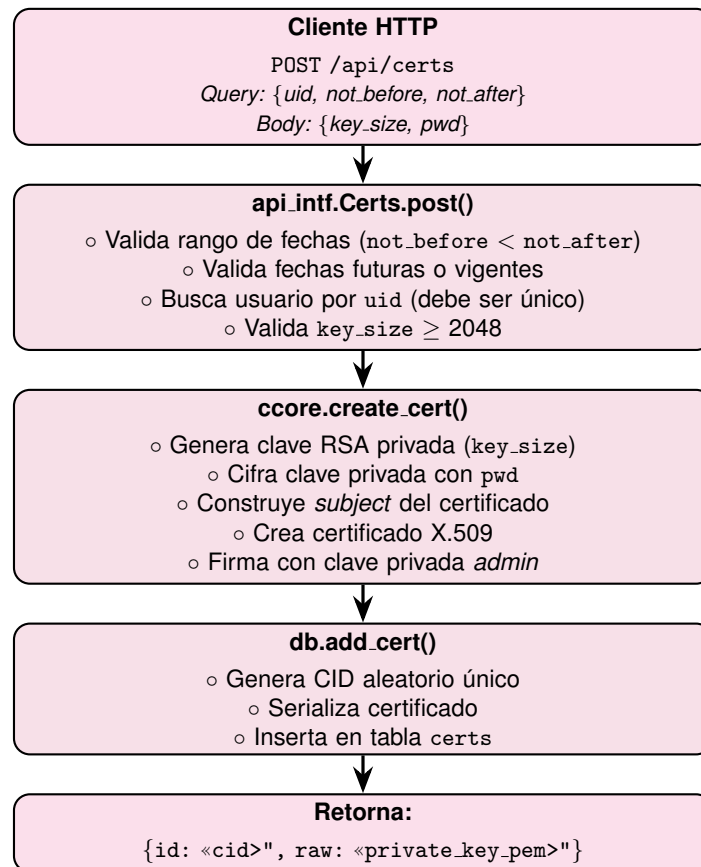


Figura 3.4: Diagrama de flujo: Generación y almacenamiento de certificados criptográficos X.509 (elaboración propia).

Finalmente, la **Figura 3.5** expone mecánicamente el funcionamiento de las peticiones de lectura parametrizadas, detallando las capas secuenciales de filtrado lógico y expresiones regulares ejecutadas sobre el archivo JSON plano del servidor.

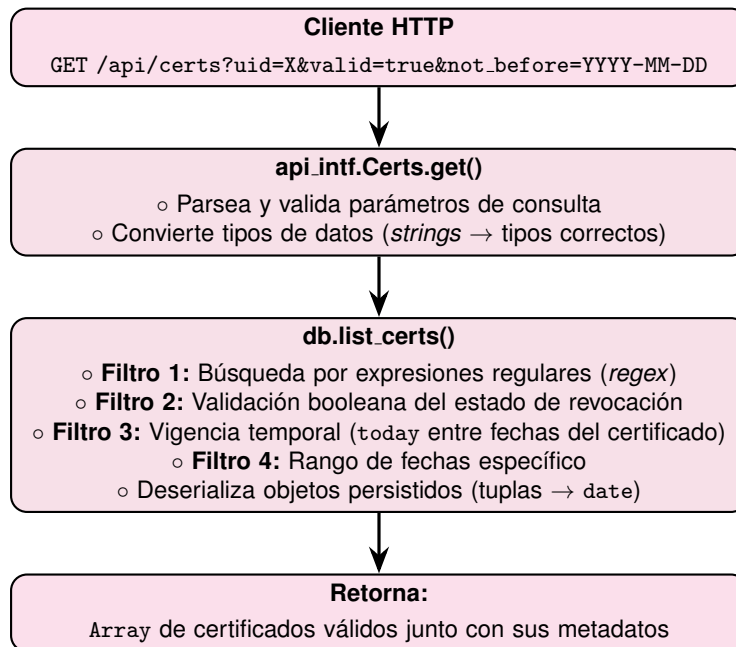


Figura 3.5: Diagrama de flujo: Filtrado y consulta de certificados criptográficos mediante la API (elaboración propia).

4. Resultados

Para garantizar que el Sistema de Gestión de Identidades cumple con los requerimientos operativos y de seguridad estipulados por la Organización Socio Formadora, se diseñó y ejecutó un plan de pruebas funcionales. El objetivo de esta fase fue comprobar la correcta interacción entre la interfaz de usuario (FrontEnd), la lógica de ruteo de la API (BackEnd) y la integridad de las operaciones criptográficas.

A continuación, se presenta la matriz de validación (Cuadro 4.1), la cual documenta los casos de prueba ejecutados, el resultado esperado y la evidencia de su funcionamiento.

Clave	Módulo	Prueba	Acción del tester / usuario	Resultado Esperado	Evidencias
01	Usuarios	Ver todos los usuarios	Consultar la lista general de usuarios.	Muestra la lista de los 8 usuarios predefinidos.	Ver enlace
02	Usuarios	Buscar usuario específico	Buscar un usuario por su nombre	Muestra únicamente la información de ese usuario.	Ver enlace
03	Usuarios	Crear usuario nuevo	Enviar los datos para crear un usuario nuevo (operativo/captura).	Responde "status": "success" devuelve el nuevo ID.	Ver enlace
04	Usuarios	Bloquear correo duplicado	Intentar crear un usuario con el ID "1"(ya existente).	Responde "status": "failure"(User ID already occupied).	Ver enlace
05	Usuarios	Validar correo electrónico	Crear un usuario con un correo sin arroba.	Error 422 indicando que el email es inválido.	Ver enlace
06	Usuarios	Consultar usuario por defecto	Pedir la información de "Mi perfil"/users/me).	Muestra el perfil default.	Ver enlace
07	Certificados	Ver todos los certificados	Consultar la lista general de certificados.	Muestra los 3 certificados predefinidos.	Ver enlace
08	Certificados	Filtrar por nivel de acceso	Buscar certificados filtrando por el nivel "Coordinador".	Muestra únicamente el certificado asociado a ese nivel.	Ver enlace
09	Certificados	Crear certificado nuevo	Enviar los datos para un certificado con un ID nuevo.	Responde "status": "success" devuelve el ID del certificado.	Ver enlace
10	Certificados	Bloquear ID duplicado	Intentar crear certificado usando el ID "2"(ya existente).	Responde "status": "failure"(Certificate ID already occupied).	Ver enlace

Cuadro 4.1: Casos de prueba y enlaces de evidencia para los módulos de Usuarios y Certificados. (elaboración propia).

Esto igualmente se incluye dentro del contexto del Manual de Usuario, en donde se detalla el paso a paso del funcionamiento de los casos.

La ejecución de las pruebas listadas en el Cuadro 4.1 confirma la viabilidad de la arquitectura propuesta. El uso del framework Flask en conjunto con la biblioteca cryptography demostró ser altamente eficiente para el procesamiento concurrente de llaves asimétricas RSA (configurables hasta 4096 bits) sin generar latencia perceptible para el usuario final.

Asimismo, la decisión de aislar las validaciones tanto en el lado del cliente (para mejorar la experiencia de usuario) como en el servidor (para evitar inyección de datos corruptos), resultó en un sistema resiliente a errores de capa humana. El manejo de la base de datos ligera (TinyDB) mantiene la integridad referencial entre las tablas de users y certs al momento de realizar consultas con filtros combinados.

5. Conclusiones y Trabajo Futuro

La elaboración de este proyecto demostró la posibilidad de diseño e implementación de un Sistema de Gestión de Identidades eficiente y adaptado a las restricciones de infraestructura de una Organización Socio Formadora. A través de una arquitectura de tres capas, se logró una plataforma centralizada que abstrae la complejidad técnica de la emisión de certificados digitales, permitiendo a su personal enfocarse en su labor humanitaria sin comprometer la seguridad de la información.

Desde una perspectiva técnica y disciplinar, el desarrollo de este reto permitió materializar los conceptos teóricos de las álgebras modernas y la criptografía en una herramienta de software funcional. La implementación de la criptografía de clave pública (asimétrica) mediante el algoritmo RSA, sustentada en la intratabilidad computacional de la factorización de números primos grandes, garantizó un canal de confianza sólido. Asimismo, el uso de funciones de resumen seguro (SHA-256), el cifrado simétrico (AES) para la protección de las llaves privadas y la estructuración bajo el estándar internacional X.509, evidenciaron cómo las matemáticas aplicadas resuelven problemas críticos de autenticidad, integridad y no repudio en los sistemas de información.

En cuanto al impacto social e institucional, la solución desarrollada trasciende el ámbito tecnológico para convertirse en un mecanismo de protección a los derechos humanos. La población migrante atendida por Casa Monarca representa un sector de extrema vulnerabilidad; Por ello, asegurar las identidades digitales de quienes manejan su información resulta imperativo. El sistema entregado apoya a la organización en el cumplimiento estricto de normativas legales mexicanas (como la Ley Federal de Protección de Datos Personales en Posesión de los Particulares).

Finalmente, si bien el sistema actual fue diseñado como un Producto Mínimo Viable (MVP) utilizando una base de datos ligera (TinyDB) e infraestructura de certificados autofirmados, su diseño modular garantiza una escalabilidad transparente. La separación de la capa de persistencia y la lógica criptográfica deja el terreno preparado para una futura migración a motores relacionales robustos y a una Infraestructura de Llave Pública (PKI) jerárquica.

Este proyecto subraya que el uso de ciencia de datos, la ingeniería de software y la criptografía son herramientas poderosas y necesarias para potenciar, proteger y dignificar la labor de las organizaciones con fin humanitario.

Ahora bien, como trabajo futuro, y partiendo de la validación exitosa del Producto Mínimo Viable (MVP) en un entorno local, el escalamiento del sistema hacia un entorno de producción demanda una serie de iteraciones estructurales y funcionales. Primeramente, a nivel de arquitectura de datos, resulta necesario migrar de la capa de persistencia actual (basada en el motor TinyDB) hacia un sistema de gestión de bases de datos relacional de grado empresarial: dicha migración garantizará el manejo adecuado de la concurrencia y la integridad transaccional ante un mayor volumen de operaciones en la ONG.

En segundo lugar, y en relación al ámbito de la ciberseguridad, con el objetivo de mitigar las vulnerabilidades inherentes al factor humano y al resguardo local de las credenciales cifradas (archivos PEM), se priorizará la integración nativa de esquemas de Autenticación Multifactor (MFA). Finalmente, se contempla la expansión de la Infraestructura de Llave Pública (PKI) base para soportar la firma automatizada de correos electrónicos y documentos internos, asegurando el no repudio en las comunicaciones de la organización.

Adicional a lo anterior, a largo plazo y de ocurrir que el proyecto continúe más allá de su fecha contemplada, la maduración de esta arquitectura podría sentar las bases tecnológicas necesarias para transicionar el control de registros hacia una red descentralizada basada en tecnología Blockchain: de este modo, se ofrecería a Casa Monarca garantías de inmutabilidad y transparencia para auditorías ante personas que muestren interés en la ONG (como autoridades gubernamentales o donantes, entre otros).

Bibliografía

- [Cámara de Diputados del H. Congreso de la Unión, 2010] Cámara de Diputados del H. Congreso de la Unión (2010). Ley Federal de Protección de Datos Personales en Posesión de Particulares. Diario Oficial de la Federación. México.
- [Chacon and Straub, 2014] Chacon, S. and Straub, B. (2014). *Pro Git*. Apress, 2nd edition.
- [Cryptography Project, 2026] Cryptography Project (2026). *Cryptography Documentation*. Accedido en mayo de 2026.
- [GitHub, Inc., 2026] GitHub, Inc. (2026). Github features and security tools. Accedido en mayo de 2026.
- [Grassi et al., 2017] Grassi, P. A., Garcia, M. E., and Fenton, J. L. (2017). Digital identity guidelines. Technical Report Special Publication (NIST SP) 800-63-3, National Institute of Standards and Technology (NIST).
- [Mohammed et al., 2025] Mohammed, A. et al. (2025). Integración continua de seguridad: Metodologías devsecops. *Software Engineering Journal*.
- [Nadipalli, 2023] Nadipalli, M. (2023). *Practical DevSecOps*. Apress.
- [OpenSSL Project, 2023] OpenSSL Project (2023). *OpenSSL Cryptography and SSL/TLS Toolkit*.
- [Oracle Corporation, 2024] Oracle Corporation (2024). *MySQL 8.0 Reference Manual*.
- [Secretaría de Economía, 2017] Secretaría de Economía (2017). Norma Oficial Mexicana NOM-151-SCFI-2016, Requisitos que deben observarse para la conservación de mensajes de datos y digitalización de documentos. Diario Oficial de la Federación. México.
- [Stallings, 2017] Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*. Pearson, 7th edition.
- [Zong, 2025] Zong, Y. (2025). Avances en algoritmos de curvas elípticas y funciones resumen. *Journal of Cryptographic Engineering*.
- [Zúñiga and Hecht, 2023] Zúñiga, A. and Hecht, F. (2023). Mecanismos de autorización dinámicos y granulares en sistemas modernos. *Revista de Seguridad Informática y Sistemas*.